

Complexity Theory (2025-26)

Major Solution (Sketches)

Name:

Roll No:

Duration: 180 Minutes

Max. Marks: 48

Instructions:

1. There are 8 questions each of 6 marks, but they are not of equal difficulty level. So read all the question.
2. Make sure that the proofs/algorithms contain all the necessary details.
3. Be precise in your answers. Writing long gibberish will not get you any marks.
4. You are allowed to use any results we have seen in the class without proving them again.

1. State one favourite open problem that you learned in this course. Indicate how you think it might be resolved in the future, with a short justification. Also describe the theoretical or practical consequences of solving this problem, if there are any. If you don't have a favourite open problem, pick any one.

Solution: This can have many possible answers, but I am choosing **P vs NP**. This problem is my favourite because it asks a very fundamental question: whether every problem whose solution can be verified efficiently can also be solved efficiently. I believe that **P** will eventually be proven to be a proper subset of **NP**, due to the fact that we have so far failed to develop a polynomial-time algorithm for any of the thousands of **NP-complete** problems, despite long efforts by many researchers.

Proving $P \neq NP$ may not have many immediate practical applications, but it will certainly have significant theoretical implications. Considering that we have not yet been able to prove $P \neq NP$ despite a substantial amount of effort, I believe the techniques that will eventually establish this result will be extremely novel and could also be used to separate other complexity classes.

Marking Scheme: It will be subjective and lenient provided that you have mentioned some open problem and written a few words on it.

2. Arrange the following complexity classes in increasing order of containment and briefly justify each containment: **P**, **NL**, **PSPACE**, **BPP**, **uniform-NC**, **PH**.

Note: Order should look like $C_1 \subseteq C_2 \subseteq C_3 \subseteq C_4 \subseteq C_5 \subseteq C_6$.

Solution: $NL \subseteq \text{uniform-NC} \subseteq P \subseteq BPP \subseteq PH \subseteq PSPACE$.

First containment is due to the fact that $NL \subseteq \text{uniform-AC}^1$ that we proved in the class. Clearly, $\text{uniform-AC}^1 \subseteq \text{uniform-AC} = \text{uniform-NC}$.

Second containment is because any problem with a **uniform-NC** circuit will be in **P** as **P** algorithm will first generate the circuit for the input and then evaluate the circuit on the input in polytime.

Third containment is because every **P** algorithm is also a **BPP** algorithm.

Fourth containment is due to Sipser-Gacs theorem.

Fifth containment is because a **PSPACE** algorithm can simulate a **PH** verifier on all the possible inputs and decide accordingly.

Marking Scheme: 3 marks for the right order, 3 for marks for every justification. Partial marks will be given accordingly.

3. Let $L = \{x \mid x \text{ is a binary string containing at least two 0s}\}$. Prove that $L \in \mathbf{AC}^0$ by constructing an appropriate circuit for L . Is $L \in \mathbf{NC}^0$? Briefly justify your answer.

Solution: To find whether x contains at least two 0s, we need to first take $\wedge$ of negation of every possible pair of input bits and then take their $\vee$. More formally, $x = x_1x_2\dots x_n$ contains at least two 0s if and only if $\vee_{1 \leq i < j \leq n} (\neg x_i \wedge \neg x_j)$. Now, this formula has a corresponding constant depth **AC** circuit.

L is not in $\mathbf{NC}^0$ as $\mathbf{NC}^0$ is a trivial class where output can depend only on a constant size fixed set of input bits which is clearly on the case for L in the problem.

Marking Scheme: 4 marks for giving an $\mathbf{AC}^0$ circuit for L with enough details. 1 mark for saying $L \notin \mathbf{NC}^0$ and 1 mark for short justification.

4. Consider the following language:

$3\text{ColExt} = \{(G, U) \mid G = (V, E) \text{ is an undirected graph, } U \subseteq V, \text{ and each}$
3-colouring of the vertices in U can be extended
to a proper 3-colouring of the entire graph $G\}$

Prove that $3\text{ColExt} \in \Pi_2^P$. Give detailed working of a Π_2^P verifier.

Note: A graph has a proper 3-colouring if every vertex is coloured using one out of 3 colours and no two neighbours have the same colour.

Solution: The Π_2^P verifier M for 3ColExt on input $((G, U), u, v)$:

- Treats u as a 3 colouring of the vertices of U and v as a 3 colouring of all the vertices of G .
- Outputs 1 if and only if:
 - v assigns the same colour to vertices of U as u assigns them.
 - v is a proper 3-colouring of the entire graph G .

M runs in polynomial time as it can perform both the tests in polynomial time. It is also clear that $(G, U) \in L$ if and only if for every u (a 3 colouring of U) there exists a v (a proper 3 colouring of G) on which M output 1.

Marking Scheme: 6 marks for the correct working of Π_2^P verifier. No proof is required if the working is clear and easy to understand. Partial marking will be done.

5. In the last lecture, while proving Sipser-Gacs theorem, for a language $L \in \mathbf{BPP}$, we proved till:

$$x \in L \iff \exists u_1, u_2, \dots, u_k \in \{0,1\}^{p(n)} \forall r \in \{0,1\}^{p(n)} r \in \cup_{i \in [k]} \{S_x \oplus u_i\}$$

Now formally put $L \in \Sigma_2^P$ by giving a polytime deterministic TM M such that

$$x \in L \iff \exists u \in \{0,1\}^{\text{poly}(n)} \forall v \in \{0,1\}^{\text{poly}(n)} M(x, u, v) = 1$$

Solution: M cannot directly check whether $r \in \cup_{i \in [k]} \{S_x \oplus u_i\}$ since generating S_x itself will take a lot of time. Instead, it will check whether for some i , $r \oplus u_i$ is in S_x . Note that, $r \oplus u_i \in S_x$ iff $M'(x, r \oplus u_i)$ outputs 1, where M' is the **BPP** machine for L . This follows from the definition of S_x . So, M will simply simulate M' on $(x, r \oplus u_i)$ and output 1 iff M' outputs 1 on $(x, r \oplus u_i)$ for some $i \in [k]$.

Marking Scheme: 6 marks for a completely correct answer. Partial marks for partially correct answers will be given depending on how close to correct answer they are.

6. A majority gate (or vertex) in a circuit is one that outputs the majority bit among the input bits. Prove that if L has an $s(n)$ -size circuit family C with $\wedge$ and $\vee$ gates with unbounded fan-in and $\neg$ gate with fan-in 1 that decides it, then L also has an $O(s(n))$ -size circuit family C' with only majority gates with unbounded fan-in and $\neg$ gate with fan-in 1 that decides it.

Solution: You need to replace $\wedge$ gates and $\vee$ gate with majority gates. Suppose an $\wedge$ gate has k inputs, then this gate can be replaced by a majority gate with $2k - 1$ input out of which k inputs are the original inputs and $k - 1$ inputs are 0s. It is easy to see now that $\wedge$ gate outputs 1 iff the corresponding majority gate outputs 1. Similarly, you can replace an $\vee$ gate with a majority gate as well. I will let you think about it. You will need at most $s(n)$ many extra gates producing 1s and 0s in the new circuit for giving input to the majority gates. Hence, the size will be $O(s(n))$.

Marking Scheme: 4 marks for replacing gates. 2 marks for justification and size analysis. Partial marks for partially correct answers will be given depending on how close to correct answer they are.

7. Prove that $\mathbf{DTIME}(n^2) \neq \mathbf{NL}$. (*Hint: You have seen a similar problem in a problem set.*)

Solution: See this <https://cs.stackexchange.com/questions/59497/prove-or-disprove-that-dtimest-nl>.

Marking Scheme: 6 marks for a completely correct answer. Partial marks for partially correct answers will be given depending on how close to correct answer they are.

8. Let's first define a new complexity class **PP**. A language L is in **PP** if there exists a polytime probabilistic Turing machine M , such that $\forall x \in \{0,1\}^*$,

$$x \in L \implies M \text{ accepts } x \text{ with probability } > 1/2$$

$$x \notin L \implies M \text{ rejects } x \text{ with probability } \geq 1/2$$

Prove that if $L \in \mathbf{PP}$, then $\bar{L} \in \mathbf{PP}$.

(**Note:** $>$ and $\geq$ signs in the definition are not a typo.)

(**Hint:** Think about the corner case where the trivial approach fails. Think about the exact probability with which a **PP** machine accepts an input. What does that number look like?)

Solution: See theorem 21.6 here <https://iccl.inf.tu-dresden.de/w/images/2/2a/CT2019-Lecture-21-print.pdf>.

Marking Scheme: 6 marks for a completely correct answer. Partial marks for partially correct answers will be given depending on how close to correct answer they are.